

Katedra informatiky
Přírodovědecká fakulta
Univerzita Palackého v Olomouci

BAKALÁŘSKÁ PRÁCE

Nástroj pro automatickou kontrolu programátorských
úkolů



2026

Vedoucí práce:
Mgr. Petr Osička, Ph.D.

My Linh Duongová

Studijní program: Informatika,
Specializace: Programování a vývoj
software

Bibliografické údaje

Autor: My Linh Duongová
Název práce: Nástroj pro automatickou kontrolu programátorských úkolů
Typ práce: bakalářská práce
Pracoviště: Katedra informatiky, Přírodovědecká fakulta, Univerzita Palackého v Olomouci
Rok obhajoby: 2026
Studijní program: Informatika, Specializace: Programování a vývoj software
Vedoucí práce: Mgr. Petr Osička, Ph.D.
Počet stran: 33
Přílohy: elektronická data v úložišti katedry informatiky
Jazyk práce: český

Bibliographic info

Author: My Linh Duongová
Title: Automated checker for programming assignments
Thesis type: bachelor thesis
Department: Department of Computer Science, Faculty of Science, Palacký University Olomouc
Year of defense: 2026
Study program: Computer Science, Specialization: Programming and Software Development
Supervisor: Mgr. Petr Osička, Ph.D.
Page count: 33
Supplements: electronic data in the storage of department of computer science
Thesis language: Czech

Anotace

Automatizované vyhodnocování programátorských úloh představuje způsob, jak zefektivnit proces hodnocení zdrojových kódů a omezit potřebu manuální kontroly v náborových procesech vývojářů. Tato bakalářská práce se zabývá návrhem a implementací modulárního systému pro automatické zpracování GitHub repozitářů, bezpečné spuštění nedůvěryhodného zdrojového kódu v izolovaném kontejnerovém prostředí a sběr výsledků vyhodnocení. Výstupem práce je funkční prototyp systému pro automatizované vyhodnocování úkolů s důrazem na bezpečnost, modularitu a snadnou rozšiřitelnost o další programovací jazyky.

Synopsis

Automated evaluation of programming assignments represents a way to streamline the source code evaluation process and reduce the need for manual review in developer recruitment processes. This bachelor's thesis deals with the design and implementation of a modular system for automatic processing of GitHub repositories, secure execution of untrusted source code in an isolated container environment and collection of evaluation results. The output of this thesis is a functional prototype of a system for automated assignment evaluation, with an emphasis on security, modularity and easy extensibility to support additional programming languages.

Klíčová slova: automatizované testování, Podman, kontejnerizace, RabbitMQ, GitHub API

Keywords: automated testing, Podman, containerization, RabbitMQ, GitHub API

TODO: Děkuji, děkuji, děkuji.

Odevzdáním tohoto textu jeho autor/ka místopřísežně prohlašuje, že celou práci včetně příloh vypracoval/a samostatně a za použití pouze zdrojů citovaných v textu práce a uvedených v seznamu literatury.

Obsah

1	Úvod	8
1.1	Analýza existujících řešení	8
1.1.1	HackerRank	8
1.1.2	CodeRunner	9
1.1.3	Judge0	9
1.1.4	Shrnutí a porovnání analýzy	9
1.2	Analýza problémů a požadavků	10
1.3	Cíl práce	11
2	Použité technologie	12
2.1	C# .NET	12
2.1.1	ASP.NET Core Web API	12
2.1.2	Entity Framework Core	12
2.1.3	Swagger (OpenAPI)	13
2.1.4	AutoMapper	13
2.1.5	Serilog	13
2.2	Podman	14
2.3	RabbitMQ	14
2.4	GitHub API	15
2.5	GitLab CI/CD	15
2.6	BASH skriptování	16
3	Architektura a návrh systému	17
3.1	Clean Architecture	17
3.2	REST API	18
3.3	Backend	19
3.4	Runners	20
4	Automatizované sestavení a nasazování	22
5	Implementace	24
5.1	Průběh zpracování požadavku	24
5.2	Zpracování výsledku	25
5.3	Datový model	25
5.4	Implementace REST API	26
6	Budoucí rozšíření	27
	Závěr	29
	Conclusions	30
A	TODO: Obsah elektronických dat	31

Seznam obrázků

1	Čistá architektura	17
2	Průběh vyhodnocení programátorské úlohy	24
3	Datový model systému InQuest	26

Seznam tabulek

1	Srovnání vlastností analyzovaných systémů	10
2	Vybrané endpointy systému	26

1 Úvod

Firma Edhouse [1], zabývající se vývojem softwaru a hardwaru, přijímá každým rokem nové stážisty. Součástí přijímacího řízení je praktické ověření znalostí, v jehož rámci společnost poskytne zájemci zadání úkolu a ten následně vypracuje své řešení ve svém preferovaném programovacím jazyce.

S rostoucím počtem uchazečů se ale tento proces setkal s kapacitními omezeními. Odevzdaná řešení byla nejednotná. Lišila se adresářovou strukturou projektů anebo verzemi použitých jazyků. Vyhodnocování úkolů vyžadovalo manuální zásah mentorů, kteří museli zdrojové kódy ručně stahovat, nastavovat lokální prostředí, řešit závislosti a kód spouštět.

Vedle vysoké časové náročnosti, která prodlužovala délku náborového procesu a odváděla vývojáře od jejich primární práce, představovalo spouštění cizího a nedůvěryhodného kódu přímo na vývojářských prostředích bez dostatečné izolace rovněž bezpečnostní riziko pro interní infrastrukturu. Odlišnosti v lokálním nastavení počítačů jednotlivých mentorů také mohly vést k nekonzistentním výsledkům při kompilaci a spouštění řešení.

Hlavní motivací pro vznik platformy **InQuest** je proto ucelená automatizace prvotní fáze vyhodnocování úkolů. Systém standardizuje proces tím, že vyžaduje sjednocenou adresářovou strukturu odevzdávaných úkolů. Dále zajišťuje autonomní stažení kódu, jeho verifikaci a především bezpečné spuštění v izolovaném kontejnerovém prostředí. Díky této automatizaci jsou mentoři odproštěni od repetitivních úkonů a mohou se soustředit na architekturu řešení, čistotu kódu a celkovou kvalitu návrhu.

1.1 Analýza existujících řešení

Automatizované vyhodnocování programátorských úloh představuje běžnou součástí náborových procesů v technologických společnostech i výuky na vysokých školách. V současnosti existuje řada komerčních i open-source systémů, které umožňují automatické sestavení, spuštění a vyhodnocení zdrojových kódů. Tyto systémy se liší zejména mírou konfigurovatelnosti, podporovanými programovacími jazyky, způsobem izolace běhového prostředí a možnostmi integrace do dalších informačních systémů.

Cílem této kapitoly je představit existující platformy, zhodnotit jejich architekturu i omezení a identifikovat vlastnosti, které byly zohledněny při návrhu systému implementovaného v této práci.

1.1.1 HackerRank

HackerRank je komerční platforma využívaná zejména při technických pohovorech, náborových akcích a online programátorských soutěžích. Uchazeč řeší zadané úlohy přímo ve webovém prostředí, přičemž systém automaticky provádí kompilaci, spuštění programu a vyhodnocení výsledků vůči připraveným testovacím sadám. [2]

Platforma podporuje desítky programovacích jazyků a nabízí pokročilé nástroje pro správu náborových procesů, tvorbu úloh i sledování výsledků kandidátů. Jedná se však o uzavřené komerční řešení, které neumožňuje vlastní implementaci pokročilé validační logiky, přímé napojení na interní repozitáře firmy ani snadné rozšíření o specifické požadavky organizace bez vysokých licenčních poplatků.

1.1.2 CodeRunner

CodeRunner je volně dostupné rozšíření pro výukový systém Moodle, které slouží k automatickému hodnocení programátorských úloh. Vyhodnocení probíhá spuštěním řešení studenta v izolovaném prostředí (sandboxu) a porovnáním výstupu s očekávanými výsledky nebo pomocí vlastních unit testů. [3]

Výhodou systému je jeho vysoká míra konfigurovatelnosti, otevřený zdrojový kód a široká podpora programovacích jazyků. Hlavní nevýhodou je však jeho úzká vazba na prostředí Moodle, což značně omezuje jeho využití v podnikovém prostředí mimo oblast elektronického vzdělávání. Systém navíc primárně zpracovává jednotlivé kódové fragmenty, nikoli ucelené projekty se složitější adresářovou strukturou.

1.1.3 Judge0

Judge0 je open-source systém poskytující robustní REST API pro kompilaci a spouštění zdrojových kódů v izolovaných Docker kontejnerech. Podporuje velké množství programovacích jazyků a je častým stavebním kamenem pro vývoj vlastních online editorů či výukových platforem. [4]

Hlavní výhodou je otevřenost projektu, modulární architektura a jednoduché rozhraní. Na druhé straně systém nabízí pouze nízkouúrovňové spouštění jednotlivých skriptů. Neposkytuje nativní podporu pro stahování celých GitHub repozitářů, automatické určování použitého jazyka, kontrolu adresářové struktury projektu ani sestavení komplexnějších aplikací využívajících balíčkovací systémy (např. NuGet, npm nebo Maven).

1.1.4 Shrnutí a porovnání analýzy

Z provedené rešerše vyplývá, že existující systémy poskytují kvalitní dílčí funkce, avšak jsou primárně navrženy buď pro akademické prostředí, nebo fungují jako uzavřené komerční platformy. Otevřená exekuční API (např. Judge0) sice nabízejí bezpečné spouštění izolovaného kódu, neřeší však ucelený proces automatického zpracování celých repozitářů odevzdaných uchazeči.

Klíčové vlastnosti a odlišnosti analyzovaných řešení jsou přehledně shrnuty v tabulce 1.

Vlastnost / Systém	HackerRank	CodeRunner	Judge0
Komerční řešení	Ano	Ne	Ne
Izolace	Uzavřený Cloud	Sandbox	Docker kontejnery
Kontrola adresáře	Ne	Ne	Ne
Vlastní zadání	Omezená	Ano	Ano
Podpora více jazyků	Ano	Ano	Ano

Tabulka 1: Srovnání vlastností analyzovaných systémů

Jak ukazují data v tabulce 1, žádný ze stávajících systémů nenabízí automatizovanou kontrolu adresářové struktury odevzdaných projektů. Komerční nástroje neumožňují přímé napojení na interní vývojářské procesy a open-source alternativy se zaměřují pouze na spouštění jednotlivých skriptů či kódových fragmentů, nikoliv ucelených projektů s vlastními závislostmi.

1.2 Analýza problémů a požadavků

Vyhodnocování úloh ve společnosti Edhouse probíhalo převážně ručně. Dalším problémem byl nejednotný způsob odevzdávání. Uchazeči zasílali svá řešení v různých formátech a s odlišnou strukturou projektu. Mentor musel odevzdané řešení stáhnout, připravit si odpovídající prostředí, projekt sestavit, spustit a následně zkontrolovat jeho výstup. Tento postup byl při větším počtu uchazečů časově náročný a zbytečně zatěžoval mentory. Bylo proto nutné stanovit jednotný způsob odevzdání, se kterým bude systém schopný dále pracovat bez ručních zásahů.

Požadavky na systém vycházely také z průběhu náborového procesu. Komunikaci s uchazeči zajišťuje především HR oddělení, proto musí být odeslání úlohy do systému co nejjednodušší. Uživatel by neměl potřebovat znalost vývojářských nástrojů ani provádět složitou konfiguraci. Základním vstupem má být pouze odkaz na odevzdaný repozitář.

Důležitou částí návrhu byla také bezpečnost. Odevzdané řešení obsahuje cizí zdrojový kód, který nelze považovat za důvěryhodný. Jeho spouštění přímo na pracovním počítači mentora by mohlo představovat riziko pro jeho stanici i interní síť. Z tohoto důvodu bylo nutné zajistit oddělené prostředí pro spuštění jednotlivých úloh. Toto prostředí musí zároveň obsahovat nástroje potřebné pro konkrétní programovací jazyk a umožnit opakovatelné spuštění za stejných podmínek.

Systém musí být připraven také na budoucí rozšiřování. Podpora programovacích jazyků se může v čase měnit, proto by přidání nového jazyka nemělo vyžadovat rozsáhlé změny v ostatních částech aplikace. Tento požadavek ovlivnil především návrh architektury a způsob oddělení částí určených pro konkrétní programovací jazyky.

Z provozního hlediska bylo požadováno, aby aplikace zvládala zpracovat více úloh současně. Vyhodnocování projektu může trvat delší dobu, a proto nesmí blokovat zpracování dalších požadavků ani nutit uživatele čekat na dokončení

kontroly. Současně musí být zajištěno, že odevzdané úlohy a přístupové údaje k repozitářům nebudou veřejně dostupné.

Z uvedených problémů vyplynuly hlavní požadavky na nový systém: sjednocení způsobu odevzdávání, jednoduché použití, bezpečné spuštění cizího kódu, možnost paralelního zpracování a snadné rozšíření o další programovací jazyky.

1.3 Cíl práce

Cílem bakalářské práce je navrhnout a implementovat systém InQuest pro automatizované vyhodnocování programátorských úloh. Systém má přijmout odkaz na odevzdaný GitHub repozitář, zjistit použitý programovací jazyk, spustit řešení v izolovaném prostředí a uložit výsledek jeho vyhodnocení.

Navržené řešení má omezit množství ruční práce mentorů, sjednotit způsob odevzdávání úloh a umožnit zadání požadavku i pracovníkům HR oddělení bez nutnosti používat vývojářské nástroje. Zpracování úloh má probíhat asynchronně, aby delší sestavení nebo spuštění jednoho projektu neblokovalo ostatní požadavky.

Součástí práce je také návrh modulární architektury, která umožní přidávat podporu dalších programovacích jazyků bez výrazných zásahů do hlavní logiky aplikace. Výstupem práce je funkční prototyp systému ověřující navržený způsob zpracování programátorských úloh.

2 Použité technologie

Při návrhu systému bylo nutné zvolit technologie, které umožní bezpečné spouštění cizího zdrojového kódu, asynchronní komunikaci mezi jednotlivými komponentami a snadné rozšiřování o podporu dalších programovacích jazyků. Tato kapitola představuje použité technologie, důvody jejich výběru a jejich roli v navrženém systému.

2.1 C# | .NET

Pro implementaci backendu byl vybrán jazyk C# na platformě .NET 8. Hlavními důvody byly podpora objektově orientovaného návrhu, silný typový systém, dostupnost nástrojů pro tvorbu webových API a vhodnost pro vrstvenou a modulární architekturu. Platforma .NET současně nabízí dobrou podporu asynchronního programování, dependency injection a běhu aplikací v kontejnerovém prostředí. [5]

Při návrhu bylo možné využít také jiné technologie, například Java se Spring Boot nebo JavaScript s prostředím Node.js.

Významnější použité knihovny a frameworky .NET jsou popsány v následujících podsekcích.

2.1.1 ASP.NET Core Web API

ASP.NET Core Web API je framework určený pro vývoj webových služeb typu REST na platformě .NET. Umožňuje vytvářet lehká, rychlá a škálovatelná rozhraní HTTP, prostřednictvím kterých mohou mezi sebou komunikovat jednotlivé aplikace nezávisle na použité technologii.

Pro tento projekt byl framework zvolen zejména kvůli přímé integraci s platformou .NET, podpoře asynchronního zpracování požadavků (*async/await*) a vestavěnému mechanismu vkládání závislostí (*dependency injection*). Důležitá byla také možnost snadno vytvářet a dokumentovat REST endpointy a oddělit prezentační vrstvu od aplikační logiky.

Rovněž podporuje směrování požadavků (*routing*), middleware, práci s chybovými stavy a zabezpečení. Pomocí Swagger UI (Swagger, viz podkapitolu 2.1.3) lze automaticky vytvořit dokumentaci jednotlivých endpointů a zároveň je během vývoje přímo testovat [6].

2.1.2 Entity Framework Core

Pro přístup k datům a správu databázové vrstvy byl zvolen framework Entity Framework Core (EF Core). Jedná se o objektově-relační mapovač (ORM), který vytváří mezivrstvu mezi objektově orientovaným kódem v C# a relační databází. Umožňuje pracovat s databázovými záznamy prostřednictvím tříd a objektů, čímž omezuje potřebu ručního vytváření SQL dotazů.

EF Core byl pro tento projekt zvolen především kvůli přímé integraci s platformou .NET a možnosti propojit databázové modely s ostatními částmi aplikace. Pro přístup k databázi využívá třídu `DbContext` a podporuje dotazování pomocí LINQ (*Language Integrated Query*). Zapsané dotazy jsou následně převedeny do SQL příkazů odpovídajících použitému databázovému systému.

Alternativou bylo ruční vytváření SQL dotazů nebo použití jednodušší knihovny, například `Dapper`. Vzhledem k malému počtu entit by bylo možné využít také tento přístup, EF Core však lépe odpovídal požadavku na správu databázového schématu a jeho společné verzování se zdrojovým kódem aplikace. [8]

V rámci vývoje byl použit přístup *Code First*, při kterém je databázové schéma definováno pomocí tříd v aplikačním kódu. Změny v datovém modelu jsou následně přenášeny do databáze pomocí migrací [7].

2.1.3 Swagger (OpenAPI)

Swagger představuje nástroj určený pro generování dokumentace REST API (viz podkapitolu 3.2) podle specifikace OpenAPI. Na základě definovaných kontrolerů a datových modelů vytváří přehlednou dokumentaci jednotlivých endpointů, jejich vstupních parametrů, návratových hodnot i možných chybových stavů.

V projektu byl použit pro automatické vytvoření dokumentace API bez nutnosti jejího ručního udržování. Webové rozhraní Swagger UI zároveň umožnilo jednotlivé endpointy během vývoje přímo volat a kontrolovat jejich odpovědi bez použití externích aplikací, například nástroje Postman. Výsledkem je dokumentace určená jak pro vývoj a testování, tak pro případné napojení dalších aplikací na vytvořené REST API. [9]

2.1.4 AutoMapper

AutoMapper je knihovna určená pro automatické mapování objektů mezi různými datovými modely. Nejčastěji se využívá při převodu databázových entit na datové přenosové objekty (angl. Data Transfer Object, dále jen DTO) a naopak.

Bez využití mapovací knihovny je nutné každou vlastnost objektu při každém převodu přiřazovat ručně, což vede ke vzniku značného množství redundantního kódu. AutoMapper tento proces pro vývoj automatizuje na základě předem definovaných mapovacích profilů, ve kterých jsou stanovena pravidla převodu mezi jednotlivými třídami. [10]

2.1.5 Serilog

Pro strukturované zaznamenávání diagnostických událostí během vývoje byla využita knihovna Serilog. Knihovna je nakonfigurována pro výstup do konzole (*Console Sink*), kde zajišťuje přehledné a barevně odlišené formátování logů včetně zachování datových struktur požadavků, což usnadňuje ladění a sledování běhu API v průběhu vývoje. Umožňuje zaznamenávat různé úrovně závažnosti událostí, jako jsou informace, varování nebo chyby. [11]

2.2 Podman

Vzhledem k tomu, že systém pracuje s nedůvěryhodnými zdrojovými kódy od externích uchazečů, bylo nutné zajistit jejich bezpečné spouštění v izolovaném prostředí. Z tohoto důvodu byla zvolena kontejnerizace, která odděluje běh testované aplikace od hostitelského operačního systému.

Při návrhu byly zvažovány nástroje Docker a Podman. Ačkoli jsou si oba nástroje z hlediska spouštění kontejnerů a syntaxe příkazů či `Dockerfile` velmi blízké, pro tento projekt byl zvolen Podman. Hlavním důvodem byla jeho *daemonless* architektura a podpora *rootless* režimu [13]. Na rozdíl od Dockeru nevyžaduje Podman centrální službu běžící na pozadí a kontejnery spouští jako běžné procesy bez administrátorských oprávnění. V případě potenciálního úniku ze spuštěného kódu v kontejneru tak útočník nezíská správu nad hostitelským systémem. [14]

Pro bezpečné předávání citlivých informací využívá Podman funkci *Podman Secrets*. Ta umožňuje ukládat tajné hodnoty (např. přístupové tokeny) mimo samotný kontejnerový obraz. Secret je při spuštění připojen jako dočasný soubor přístupný pouze běžícímu kontejneru a nestává se součástí proměnných prostředí. [15]

Další klíčovou výhodou Podmanu je možnost řízení kontejnerů přes jeho nativní REST API (Libpod API). Backendová aplikace tak může prostřednictvím HTTP rozhraní automaticky vytvářet nové kontejnery, předávat jim vstupní parametry, sledovat jejich stav a po dokončení vyhodnocení zajišťovat jejich bezpečné smazání. [16]

2.3 RabbitMQ

Pro předávání výsledků mezi izolovanými kontejnery a backendovou aplikací byl zvolen zprávový broker RabbitMQ. Jeho hlavním úkolem je zajistit asynchronní komunikaci mezi komponentami, které nemusejí být spuštěny ani dokončit svou činnost ve stejném okamžiku.

Po dokončení vyhodnocení publikuje kontejner zprávu s výsledkem do příslušné fronty. Backendová aplikace tuto zprávu následně asynchronně přijme, zpracuje a uloží získaná data do databáze. Díky tomu backend nemusí po spuštění kontejneru čekat na dokončení celé validace a může mezitím zpracovávat další požadavky.

Alternativou bylo pravidelné dotazování backendu na stav kontejneru nebo přímé odeslání výsledku prostřednictvím HTTP požadavku. Pravidelné dotazování by však vytvářelo zbytečné požadavky a přímá HTTP komunikace by vyžadovala, aby byl backend v okamžiku dokončení úlohy dostupný. RabbitMQ byl proto zvolen kvůli možnosti ukládat zprávy do fronty a oddělit životní cyklus kontejneru od zpracování výsledku v backendové aplikaci.

Další alternativou ke zprávovému brokeru RabbitMQ mohla být platforma Apache Kafka. Během návrhu systému byla tato možnost zvažována, protože Kafka je vhodná pro zpracování velkých proudů událostí, jejich dlouhodobé ucho-

vávání a opakované čtení více konzumenty. Pro potřeby systému InQuest by však její použití představovalo zbytečně složité řešení, jelikož komunikace mezi kontejnery a backendem je založena především na předávání jednotlivých výsledků vyhodnocení. Z tohoto důvodu byl nakonec zvolen RabbitMQ, který lépe odpovídá požadavku na frontové zpracování zpráv, potvrzování jejich doručení a jednodušší oddělení jednotlivých komponent systému.

Použití zprávového brokeru současně vytváří volnou vazbu (*loose coupling*) mezi jednotlivými komponentami systému. Dočasné zvýšení počtu zpracovávaných úloh tak nemusí okamžitě zatížit backend, protože výsledky mohou být postupně odebírány ze zprávové fronty. [17]

2.4 GitHub API

Pro získávání informací o odevzdaných repozitářích bylo použito GitHub API. Jedná se o RESTové rozhraní umožňující získávat informace o repozitářích, uživateli, větvích a jejich obsahu. [20]

V systému InQuest se GitHub API používá především pro ověření existence a dostupnosti zadaného repozitáře a pro zjištění jeho hlavního programovacího jazyka. Na základě této informace backend vybere odpovídající kontejnerový runner, ve kterém bude řešení zpracováno.

Komunikace probíhá pomocí osobního přístupového tokenu (*Personal Access Token*). Token umožňuje přístup k neveřejným repozitářům, které jsou při odevzdávání stážistických úloh vyžadovány. Současně zvyšuje limit počtu požadavků oproti anonymnímu přístupu.

Alternativou bylo požadovat, aby programovací jazyk zadával uživatel ručně společně s adresou repozitáře. Tento přístup by však zvyšoval počet vstupních údajů a mohl vést k nesouladu mezi zvoleným jazykem a skutečným obsahem projektu. Automatické získání informace prostřednictvím GitHub API proto zjednodušuje použití systému zejména pro pracovníky HR oddělení.

2.5 GitLab CI/CD

Pro automatizaci sestavení a nasazení systému bylo použito GitLab CI/CD. Jedná se o nástroj integrovaný do platformy GitLab, který umožňuje spouštět předem definované úlohy při změnách zdrojového kódu. Jednotlivé úlohy jsou vykonávány pomocí GitLab Runnerů na odděleném serveru nebo v kontejnerovém prostředí.

V projektu GitLab CI/CD zajišťuje sestavení backendové aplikace, vytvoření jednotlivých kontejnerových obrazů a jejich nahrání do registru. Automatizace byla důležitá zejména kvůli většímu počtu runnerů určených pro různé programovací jazyky. Jejich ruční sestavování a publikování by bylo časově náročné a mohlo by vést k rozdílům mezi jednotlivými verzemi. [18]

Alternativou bylo například použití GitHub Actions nebo samostatného nástroje Jenkins. GitLab CI/CD byl zvolen především proto, že zdrojový kód pro-

jektu i registr kontejnerových obrazů byly již spravovány v prostředí GitLab v rámci firmy. Nebylo proto nutné zavádět další službu ani řešit propojení mezi více platformami.

Použití pipeline odstraňuje většinu manuálních kroků při sestavení a poskytuje okamžitou informaci o tom, zda se jednotlivé části systému podařilo úspěšně vytvořit.

2.6 BASH skriptování

Pro řízení kroků uvnitř kontejnerových runnerů a pro automatizaci částí CI/CD pipeline byly použity skripty v jazyce BASH. BASH (*Bourne Again Shell*) je příkazový interpret a skriptovací jazyk běžně dostupný v linuxových prostředích. [19]

V projektu skripty zajišťují například kontrolu vstupních proměnných, stažení repozitáře, ověření struktury projektu, spuštění kompilace a předání výsledku zpět backendové aplikaci. V CI/CD pipeline jsou využity také pro dynamické vyhledávání dostupných runnerů a sestavení jejich kontejnerových obrazů.

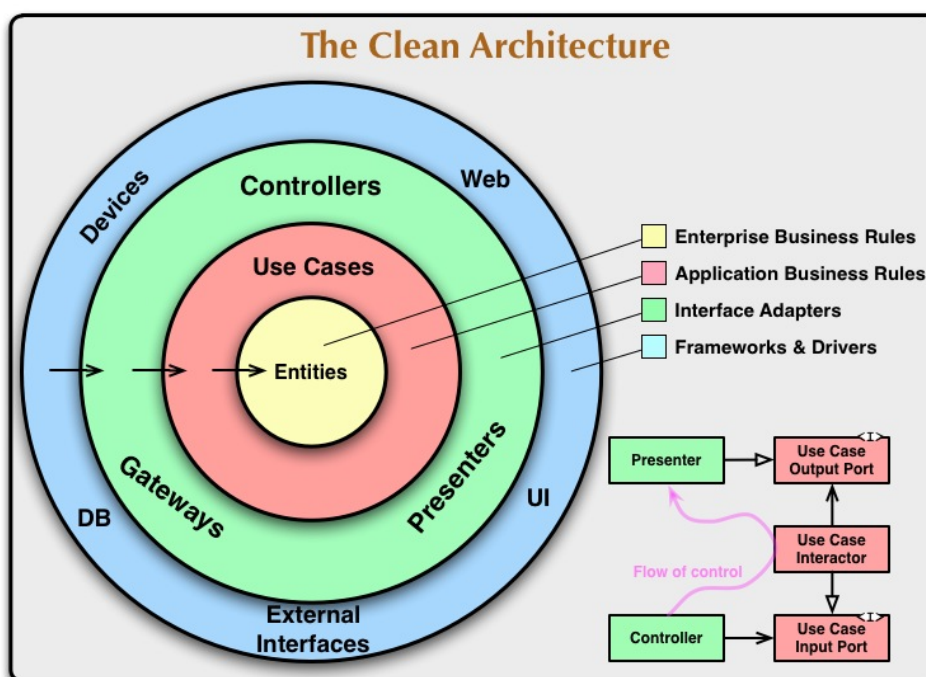
Pro práci s konfiguračními a textovými soubory byly využity standardní nástroje, například `sed` a `xmlstarlet`. Výhodou tohoto řešení je jejich dostupnost přímo v linuxových kontejnerech bez nutnosti vytvářet další pomocnou aplikaci. Nevýhodou je složitější údržba rozsáhlejších skriptů, proto jsou jednotlivé části rozděleny podle jejich odpovědnosti do samostatných souborů.

3 Architektura a návrh systému

Na základě požadavků popsanych v předchozí kapitole bylo navrženo řešení umožňující automatizované vyhodnocování programátorských úloh. Systém musí zajistit bezpečné spuštění nedůvěryhodného kódu, podporovat více programovacích jazyků, komunikovat s externími službami a ukládat výsledky jednotlivých vyhodnocení. Současně bylo cílem navrhnout architekturu umožňující jednoduché rozšíření o podporu dalších programovacích jazyků a další validační pravidla bez rozsáhlých změn v hlavní aplikační logice.

Z těchto důvodů byla backendová aplikace navržena podle principů Clean Architecture, která odděluje aplikační a doménovou logiku od databáze, webového rozhraní a externích služeb. Změna konkrétní infrastrukturní technologie tak nemusí přímo ovlivnit hlavní logiku systému.

3.1 Clean Architecture



Obrázek 1: Čistá architektura

Clean Architecture, česky „Čistá architektura“ je návrhový princip, který definuje strukturu softwarového systému tak, aby byla nezávislá na frameworku, UI, databázi a externích agentech. Díky oddělení byznysové logiky a technických detailů zajišťuje snadnou údržbu, testovatelnost a abstrakci mezi vrstvami.

Podle striktního pravidla závislosti (angl. The Dependency Rule) je směr vrstev definován do středu. Vnitřní vrstvy nemají povědomí o tom, co se děje ve vnějších vrstvách. [21]

Pro systém InQuest byla Clean Architecture zvolena především kvůli potřebě oddělit proces vyhodnocování úloh od konkrétních infrastrukturních nástrojů. Aplikační logika tak není přímo závislá například na implementaci databáze, službě Podman, zprávovém brokeru RabbitMQ nebo GitHub API. Toto oddělení usnadňuje změnu jednotlivých technologií a další rozšiřování systému.

Architektura se skládá z následujících vrstev:

- **Doménová** (*Domain*) se nachází v samotném jádře architektury. Obsahuje základní doménové modely, výjimky a pravidla, která nejsou závislá na databázi, webovém rozhraní ani externích knihovnách.
- **Aplikační** (*Application*) obsahuje případy užití systému a řídí průběh jednotlivých operací. V systému InQuest zajišťuje například vytvoření požadavku na vyhodnocení, validaci vstupních údajů a zpracování přijatého výsledku.
- **Infrastrukturní** (*Infrastructure*) implementuje komunikaci s databází a externími službami. Patří sem například přístup k databázi pomocí Entity Framework Core, řízení kontejnerů přes Podman, komunikace s RabbitMQ a získávání údajů prostřednictvím GitHub API.
- **Prezentační** (*Presentation*): vystavuje koncové body pro komunikaci přes HTTP. Příjímá požadavky klienta, předává je aplikační vrstvě a převádí výsledky zpracování do odpovídajících HTTP odpovědí.

Konkrétní rozdělení backendové aplikace do uvedených vrstev a odpovědnosti jednotlivých projektů jsou popsány v podkapitole [3.3](#).

3.2 REST API

Pro komunikaci klientských aplikací se systémem bylo zvoleno REST API. Tento způsob komunikace umožňuje vystavit funkce backendu prostřednictvím HTTP rozhraní a oddělit tak implementaci serverové části od konkrétního uživatelského rozhraní. Systém může být díky tomu používán například prostřednictvím Swagger UI nebo být napojen na další interní aplikace společnosti.

REST (*Representational State Transfer*) je architektonický styl určený pro návrh distribuovaných webových služeb. Komunikace probíhá pomocí protokolu HTTP a standardních metod, jako jsou GET, POST, PUT nebo DELETE [\[12\]](#).

Jednou z vlastností REST rozhraní je bezstavovost (*stateless*). Každý požadavek musí obsahovat informace potřebné pro jeho samostatné zpracování a server nemusí mezi jednotlivými požadavky uchovávat stav klientské relace. To zjednodušuje komunikaci mezi klientem a backendem a umožňuje nezávislé zpracování požadavků.

V systému InQuest slouží REST API zejména k vytvoření nového požadavku na vyhodnocení úlohy, získání jeho aktuálního stavu, načtení uložených výsledků

a případnému opakování vyhodnocení. Samotná kontrola řešení probíhá asynchronně. API proto po přijetí požadavku nečeká na sestavení a spuštění odevzdaného projektu, ale vrátí klientovi informace o vytvořeném záznamu. Výsledek lze následně získat samostatným požadavkem.

3.3 Backend

Backendová aplikace je rozdělena do několika projektů, které odpovídají jednotlivým vrstvám Clean Architecture. Každá vrstva má vymezenou odpovědnost a s ostatními částmi komunikuje prostřednictvím definovaných rozhraní. Díky tomu lze měnit implementaci infrastruktury nebo externích služeb bez zásahu do aplikační logiky.

Prezentační vrstva zajišťuje komunikaci mezi klientem a aplikační vrstvou prostřednictvím REST API. Přijímá HTTP požadavky, provádí jejich základní kontrolu a předává je odpovídajícím aplikačním službám. Zároveň převádí výsledky zpracování do HTTP odpovědí.

V této vrstvě se registrují služby využívané prostřednictvím dependency injection a inicializují se nástroje Swagger a Serilog. Swagger poskytuje interaktivní dokumentaci REST API, zatímco Serilog slouží k zaznamenávání informací o průběhu zpracování požadavků a vzniklých chybách.

Projekt obsahuje složku `Controllers`, ve které jsou definovány jednotlivé kontrolery a jejich endpointy. Složka `Filters` obsahuje filtry určené pro zachytávání a jednotné zpracování výjimek. Převod mezi interními modely aplikace a datovými objekty vystavovanými prostřednictvím API zajišťují mapovací profily ve složce `Mapper`.

Projekt DTO představuje unifikovaný datový kontrakt pro celý systém. DTO jsou reprezentovány jako třídy bez byznys logiky, které přenášejí data mezi jednotlivými vrstvami a externími systémy.

Doménová vrstva představuje vnitřní část aplikace a obsahuje základní modely systému, abstrakce služeb, vlastní výjimky a konfigurační objekty. Tato vrstva není závislá na webovém frameworku, databázi ani konkrétních externích službách. Ostatní vrstvy mohou využívat modely a rozhraní, která jsou v ní definována. V tomto specifickém systému jsou popsány modely pro abstrakce (rozhraní služeb), výjimky, nastavení a tak.

Aplikační vrstva obsahuje orchestraci datových toků a validační logiku procesů. Jejím úkolem je například validace vstupních údajů, vytvoření požadavku na vyhodnocení a zpracování výsledku přijatého z kontejnerového prostředí. Implementace je rozdělena do několika částí:

- **/AssignmentEvaluation** obsahuje logiku spojenou se zahájením vyhodnocení úlohy. Zpracovává adresu GitHub repozitáře, ověřuje její platnost a prostřednictvím GitHub API získává informace o použitém programovacím jazyce.
- **/Services** je popis poskytovaných služeb.

- **Assignment**: spravuje životní cyklus požadavku a stav ukládá do databáze.
- **Result**: zpracovává výstup po vyhodnocení, rozdělí standardní výstup na informace o sestavení a spuštění, nastavuje příznaky úspěchu a ukládá výsledek do databáze.
- **Value**: kontroluje hodnoty získané z výstupu řešení uchazeče vůči očekávaným hodnotám. Pro každé zadání se modul mění.

- **/Validator** je složka pro služby, které se týkají validace.

Infrastrukturní vrstva zajišťuje komunikaci s databází a externími službami. Přístup k databázi je realizován pomocí *Entity Framework Core*. Komponenta `PodmanManager` komunikuje se službou Podman prostřednictvím jejího HTTP rozhraní a zajišťuje vytváření, spouštění, sledování a odstraňování kontejnerů.

Součástí infrastruktury je také napojení na *RabbitMQ*, které zajišťuje asynchronní příjem výsledků z jednotlivých kontejnerů a jejich předání aplikační vrstvě ke zpracování. Integrace s *GitHub API* slouží k ověření existence a dostupnosti repozitáře a k získání informace o jeho hlavním programovacím jazyce.

Takto navržené rozdělení odděluje hlavní logiku vyhodnocování od konkrétních technologií. Externí služby lze díky tomu upravit nebo nahradit, aniž by bylo nutné výrazně měnit případy užití definované v aplikační vrstvě.

3.4 Runners

Runnery představují jednu z nejdůležitějších částí systému InQuest. Zajišťují zpracování odevzdaných projektů, jejich sestavení a spuštění v izolovaném kontejnerovém prostředí. Zároveň řeší jeden z hlavních požadavků práce, kterým je podpora více programovacích jazyků bez nutnosti měnit hlavní logiku aplikace.

Při jejich návrhu bylo proto důležité oddělit společný průběh vyhodnocení od příkazů specifických pro jednotlivé programovací jazyky. Každý podporovaný jazyk využívá vlastní runner, jehož úkolem je ověřit strukturu odevzdaného projektu, provést jeho sestavení a následně jej spustit.

Jednotlivé runnery sdílejí společnou část implementace, která zajišťuje řízení celého procesu, práci se vstupními údaji a odeslání výsledku. Liší se pouze částí obsahující příkazy specifické pro konkrétní programovací jazyk. Díky tomuto rozdělení není nutné při přidání nového jazyka znovu implementovat celý průběh vyhodnocení.

Společná část runnerů obsahuje následující skripty:

- **checkVariables.sh** je skript, který kontroluje přítomnost požadovaných vstupních údajů při startu kontejneru. Nejprve ověří, zda byla přiložena URL adresa ke GitHub repozitáři a poté jestli má kontejner přístup k cestě pro access token v Podman secrets. Token je nutný pro práci se

soukromými repozitáři (a k funkčnosti i s veřejnými) a je do kontejneru předáván prostřednictvím mechanismu Podman Secrets.

- **generateValidationResult.sh** vytváří výstupní datovou strukturu obsahující informace o průběhu a výsledku vyhodnocení.
- **reportResult.sh** je skript, který výsledek odešle prostřednictvím RabbitMQ do zprákové fronty, odkud jej převezme backendová aplikace.
- **validate.sh** obsahuje hlavní logiku kontroly odevzdaného řešení. Zajišťuje stažení GitHub repozitáře, získání potřebných údajů o projektu a spuštění jazykově specifického skriptu `start.sh`. Běh skriptu je omezen časovým limitem, aby nemohlo dojít k nekonečnému zpracování například v důsledku zablokovaného nebo chybně implementovaného programu.
- **run.sh** představuje hlavní vstupní bod runneru. Řídí pořadí jednotlivých kroků, spouští ostatní skripty a zajišťuje ukončení procesu po dokončení nebo vzniku chyby.

Konkrétní implementace pro jednotlivé programovací jazyky jsou umístěny ve složkách se zkratkou příslušného programovacího jazyka. Každá z těchto složek využívá společné skripty a doplňuje je o příkazy potřebné pro sestavení a spuštění projektu daného jazyka.

Každá jazyková složka obsahuje zejména následující soubory:

- **runnerScript-<language_extension>.sh** reprezentuje konkrétní implementaci daného jazyka. Ověřuje přítomnost povinných souborů v projektu a spouští odpovídající příkazy technologie ke kompilaci a spuštění běhu.
- **Dockerfile** definuje předpis pro sestavení kontejnerového obrazu runneru. Přibalí se zde sdílené skripty a stáhnou se dodatečné systémové příkazy podle skriptu `../installPrerequisites.sh`. Jazykově specifický skript je při sestavení přejmenován na `start.sh` a jako vstupní bod kontejneru je nastaven skript `run.sh`.

Tento návrh odděluje společný průběh vyhodnocení od příkazů závislých na konkrétním programovacím jazyce. Přidání podpory nového jazyka proto spočívá především ve vytvoření nové složky, definici odpovídajícího kontejnerového obrazu a implementaci jazykově specifického skriptu.

Postup vytvoření runneru pro nový programovací jazyk je podrobně popsán v dokumentaci `testRunnerDocumentation.md`, který se nachází v adresáři `./documentations`.

4 Automatizované sestavení a nasazování

Jelikož aplikace využívá větší množství kontejnerových obrazů a podporuje postupné rozšiřování o nové programovací jazyky, bylo potřeba automatizovat proces sestavení a nasazení. Z tohoto důvodu byla vytvořena CI/CD pipeline v prostředí GitLab. Konfigurace je definována v souboru **.gitlab-ci.yml**. Pipeline zajišťuje automatizované verzování, kompilaci backendu, přípravu Docker obrazů a dynamické sestavení izolovaných testovacích runnerů.

Pipeline je rozdělena do následujících fází, které nesou v názvu příponu `__job`:

- **update**: tato fáze přiděluje každému sestavení unikátní číslo verze odpovídající historii v GitLabu. Skript nejdříve vyparsuje základní verzi ze souboru `Directory.Build.props`. K této verzi se připojí identifikátor běžící pipeline. Výsledné číslo verze (např. 1.0.67) je pomocí **xmlstarlet** zpětně zapsáno do atributů *AssemblyVersion* a *PackageVersion* a následně předáno jako artefakt souboru *version.env* pro další úlohy. Díky tomu používají všechny sestavené části systému stejné číslo verze.
- **build** a **deploy**: zde se na základě definovaného `Dockerfile` sestaví obrazy aplikace s tagem unikátního čísla verze získaného z předchozího kroku a s tagem `latest`. Tyto obrazy jsou poté ve fázi **deploy** nahrány do kontejnerového registru, odkud je lze následně stáhnout na cílový server a použít při nasazení aplikace.
- **build-test-runners** a **deploy-test-runners**: stejně jako fáze **build**, tato také sestavuje obrazy pro jednotlivé podporované programovací jazyky. Protože je systém navržen tak, aby bylo možné přidat nový jazyk vytvořením nové složky v adresáři `runners`, byla obdobným způsobem navržena také CI/CD pipeline. Provádí následující operace:
 1. Skript projde všechny podadresáře v adresáři `./runners/`. Každý nalezený podadresář představuje implementaci runneru pro konkrétní programovací jazyk, například `cs`, `cpp` nebo `java`.
 2. Pro každý runner je načten příslušný soubor `Dockerfile`. Pomocí nástroje `sed` je z instrukce `FROM ... AS final` získána verze použitého základního obrazu.
 3. Na základě názvu jazyka a získané verze je sestaven tag výsledného obrazu, například `test-runner-cs:8.0`.
 4. Pro každý nalezený jazyk je spuštěno sestavení kontejnerového obrazu.
 5. Dynamicky sestaví tag obrazu (např. `test-runner-cs:8.0`) a spustí `docker build`.
 6. Seznam nově vytvořených obrazů je uložen a předán další úloze, která zajistí jejich hromadné odeslání do registru.

Současná CI/CD pipeline zajišťuje verzování, sestavení a publikování backendové aplikace a kontejnerových runnerů. V konfiguraci je připravena také struktura pro fázi automatického testování, tato část zatím není implementována. V budoucnu by bylo vhodné pipeline rozšířit o spouštění jednotkových a integračních testů před sestavením a publikováním výsledných obrazů. Tím by bylo možné zachytit chyby před jejich nasazením a zabránit publikování nefunkční verze systému.

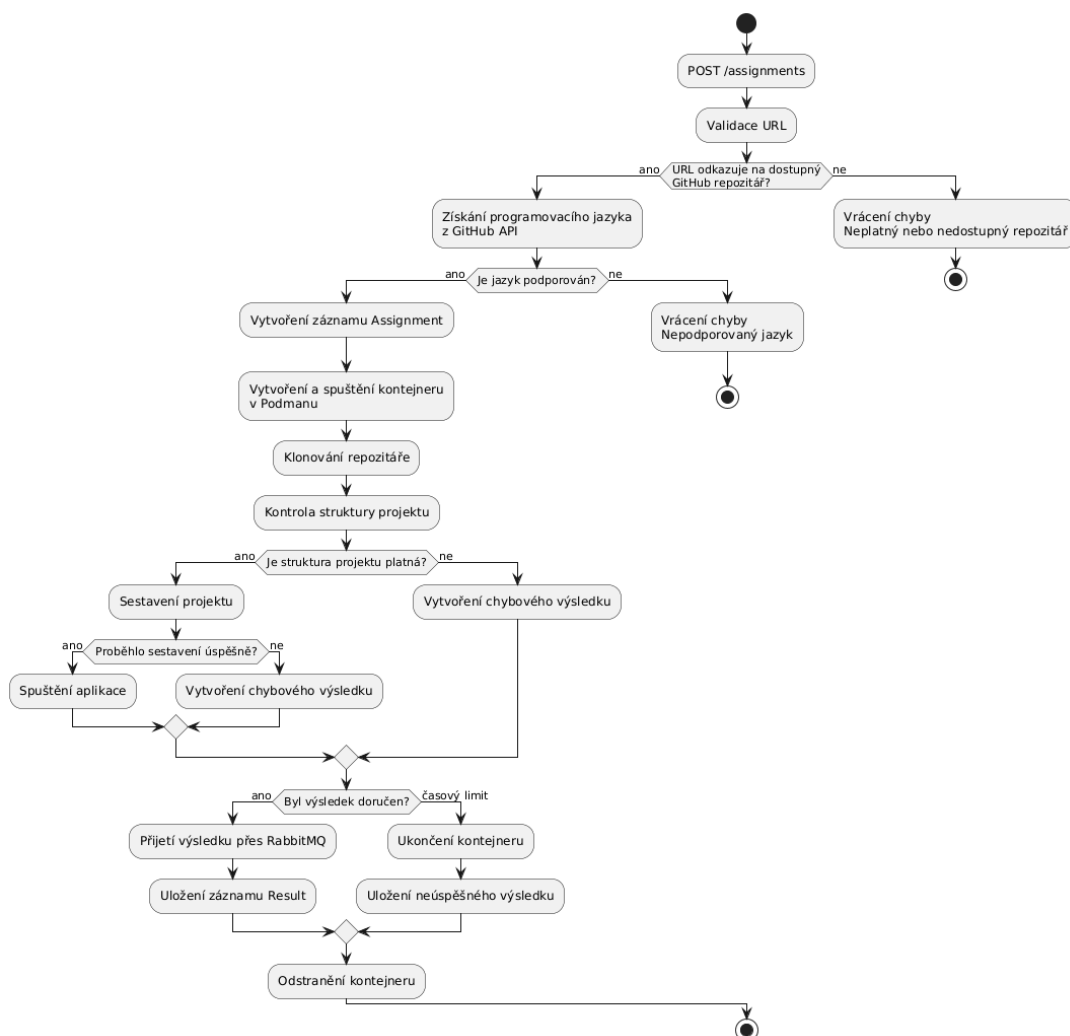
Pipeline tedy neobsahuje pevně zapsaný seznam podporovaných jazyků, ale zjišťuje jej přímo ze struktury adresáře `runners`. Tento přístup snižuje režii spojenou s údržbou systému. Při nasazení podpory pro nový programovací jazyk není potřeba upravovat konfiguraci CI/CD pipeline, protože skript novou složku najde a zpracuje autonomně.

5 Implementace

Tato kapitola se věnuje průběhu hlavního procesu systému InQuest, který začíná přijetím požadavku na vyhodnocení a končí uložením výsledku do databáze. Zaměřuje se na spolupráci backendové aplikace, GitHub API, kontejnerového prostředí Podman a zprávového brokeru RabbitMQ. Návrh jednotlivých komponent a jejich rozdělení podle Clean Architecture byl popsán v předchozí kapitole.

5.1 Průběh zpracování požadavku

Celkový průběh vyhodnocení programátorské úlohy je znázorněn na obrázku 2. Proces začíná přijetím požadavku typu POST, který obsahuje adresu GitHub repozitáře s odevzdaným řešením.



Obrázek 2: Průběh vyhodnocení programátorské úlohy

Proces začíná přijetím požadavku typu POST, který obsahuje adresu GitHub repozitáře. Backend ověří formát adresy a pomocí GitHub API zkontroluje dostupnost repozitáře a zjistí jeho hlavní programovací jazyk. Pokud repozitář není dostupný nebo jazyk není podporován, zpracování je ukončeno odpovídající chybovou odpovědí.

Pro platný požadavek je v databázi vytvořen záznam typu *Assignment*. Na základě zjištěného programovacího jazyka je poté v Podmanu vytvořen a spuštěn kontejner s odpovídajícím runnerem. Runner naklonuje repozitář, zkontroluje jeho strukturu a pokusí se projekt sestavit a spustit. Pokud některý z těchto kroků selže, je vytvořen výsledek obsahující informace o vzniklé chybě.

Po dokončení vyhodnocení je výsledek odeslán prostřednictvím RabbitMQ zpět backendové aplikaci, která jej uloží do záznamu typu *Result*. Pokud kontejner výsledek v nastaveném časovém limitu neodešle, je ukončen a do databáze je uložen neúspěšný výsledek. V obou případech je kontejner po dokončení procesu odstraněn.

5.2 Zpracování výsledku

Po dokončení validace vytvoří runner výsledek obsahující informace o průběhu kontroly, sestavení a spuštění projektu. Součástí výsledku jsou také zaznamenané standardní a chybové výstupy jednotlivých kroků. Vytvořená zpráva je následně odeslána prostřednictvím RabbitMQ do fronty, ze které ji asynchronně přijímá backendová aplikace.

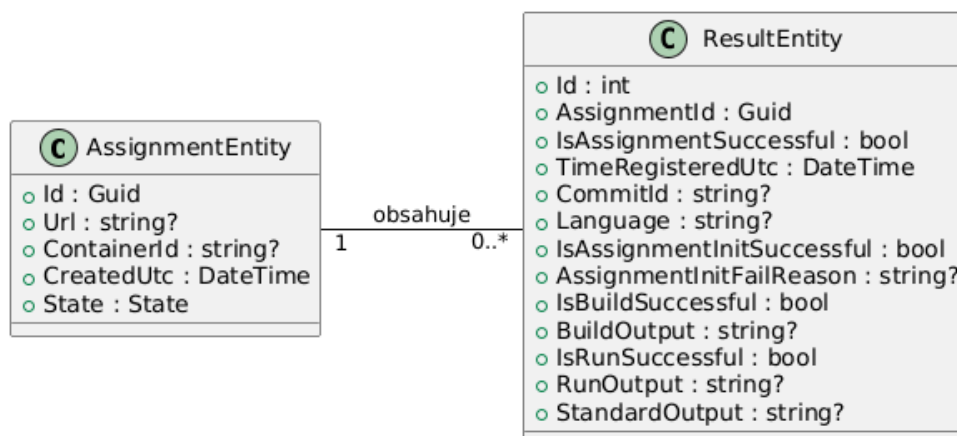
Po přijetí zprávy backend vyhledá odpovídající záznam typu *Assignment* podle předaného identifikátoru. Přijatá data zpracuje, rozdělí výstupy na část týkající se sestavení a spuštění projektu a určí výsledný stav vyhodnocení. Následně vytvoří záznam typu *Result*, který uloží do databáze a přiřadí jej ke konkrétnímu zadání.

Systém musí zohlednit také situaci, kdy runner výsledek neodešle. Může k tomu dojít například při zablokování programu, nekonečné smyčce nebo neočekávaném ukončení procesu. Backend proto pravidelně kontroluje dobu běhu aktivních kontejnerů. Pokud je překročen stanovený časový limit, kontejner je ukončen a odstraněn. Do databáze je následně uložen neúspěšný výsledek s informací o překročení časového limitu.

5.3 Datový model

Datový model systému je tvořen dvěma hlavními entitami *AssignmentEntity* a *ResultEntity*. Jejich vztah je znázorněn na obrázku 3.

Entita *AssignmentEntity* představuje jeden požadavek na vyhodnocení programátorské úlohy. Obsahuje identifikátor, adresu GitHub repozitáře, identifikátor spuštěného kontejneru, čas vytvoření požadavku a jeho aktuální stav. Identifikátor kontejneru je využíván zejména při sledování běžících vyhodnocení a při následném odstranění kontejneru.



Obrázek 3: Datový model systému InQuest

Entita *ResultEntity* uchovává výsledek konkrétního vyhodnocení. Obsahuje informace o úspěšnosti inicializace, sestavení a spuštění projektu, zaznamenané výstupy jednotlivých kroků, použitý programovací jazyk a identifikátor zpracovaného commitu. Atribut *AssignmentId* slouží jako cizí klíč odkazující na příslušný záznam *AssignmentEntity*.

Mezi entitami je vytvořena vazba jedna k mnoha. Jeden záznam *AssignmentEntity* může obsahovat nula až více záznamů *ResultEntity*. Díky tomu lze ke stejnému zadání ukládat výsledky opakovaných vyhodnocení a zachovat jejich historii.

5.4 Implementace REST API

REST API zpřístupňuje hlavní případy užití systému klientským aplikacím. Nejdůležitější operace jsou uvedeny v tabulce 2.

Metoda	Endpoint	Význam
POST	/api/Assignments	Registrace požadavku na vyhodnocení
GET	/api/Assignments	Získání všech uložených zadání
GET	/api/Assignments/{id}	Získání detailu konkrétního zadání
PUT	/api/Assignments/{id}	Opakované vyhodnocení uloženého zadání
DELETE	/api/Assignments/{id}	Odstranění zadání z databáze

Tabulka 2: Vybrané endpointy systému

6 Budoucí rozšíření

Navržený systém byl vytvořen s důrazem na modularitu a snadnou rozšiřitelnost. Současná implementace splňuje stanovené požadavky, nicméně architektura umožňuje další rozvoj, který může zvýšit kvalitu systému, usnadnit jeho údržbu a rozšířit možnosti jeho využití.

- **Náhrada služby GitHub Classroom:** V průběhu dokončování práce bylo oznámeno ukončení služby GitHub Classroom k 28. srpnu 2026. GitHub Classroom byl ve společnosti využíván pro vytváření soukromých repozitářů, do kterých uchazeči odevzdávali vypracované úlohy.

Ukončení této služby nemá přímý vliv na hlavní funkce systému InQuest, protože systém pracuje s běžnými GitHub repozitáři prostřednictvím jejich URL a GitHub API. V budoucnu ale bude nutné nahradit způsob, kterým jsou repozitáře pro jednotlivé uchazeče automaticky vytvářeny, nastavovány jako soukromé a předávány účastníkům náborového procesu. Samotný proces vyhodnocení může zůstat beze změny, pokud nové řešení zachová odevzdávání úloh prostřednictvím GitHub repozitářů.

- **Možnost nastavení očekávaného výsledku:** Současná implementace používá validační pravidla definovaná přímo v systému. V budoucnu by bylo vhodné umožnit zadavateli úlohy nastavit očekávaný výsledek, podle kterého má být odevzdané řešení vyhodnoceno. Tím by bylo možné systém využít pro větší množství různých zadání bez nutnosti upravovat validační logiku v aplikačním kódu.
- **Automatizované testování v CI/CD:** Současná CI/CD pipeline zajišťuje sestavení aplikace a vytvoření kontejnerových obrazů. V budoucnu by bylo vhodné pipeline rozšířit o fázi, která zajišťuje automatické spouštění testů před samotným sestavením a nasazením aplikace. Díky tomu by bylo možné zachytit případné chyby během vývojového procesu a zabránit nasazení nefunkčních verzí systému.
- **Podpora dalších programovacích jazyků:** Architektura systému byla navržena tak, aby bylo možné jednoduše přidávat podporu dalších programovacích jazyků. Rozšíření spočívá především ve vytvoření nového runneru obsahujícího odpovídající kontejnerový obraz a skript definující proces sestavení a spuštění projektu. V současné verzi systém podporuje jazyky C#, C++, Java, Python, Rust, JavaScript a TypeScript.
- **Rozšíření REST API a správa dat:** Stávající REST rozhraní se zaměřuje na základní životní cyklus jednotlivých požadavků. Pro usnadnění správy a integraci s rozsáhlejšími systémy by bylo v budoucnu vhodné API rozšířit o nové endpointy:

- **Hromadné zpracování požadavků (POST /api/batch):** Umožnilo by odeslat pole odkazů na repozitáře v jediném HTTP požadavku rozhraní. Systém by následně hromadně vytvořil frontu úloh pro spuštění v runneru, což by zjednodušilo obsluhu pro větší počet úloh naráz.
- **Správa a promazávání dat (DELETE /api/all):** Administrátorský endpoint určený k hromadnému mazání historických záznamů a výsledků (případně s možností filtrů podle data či stavu). To by usnadnilo údržbu databáze a uvolňování místa na disku.

Závěr

Cílem bakalářské práce bylo navrhnout a implementovat systém InQuest pro automatizované vyhodnocování programátorských úloh. Navržené řešení přijímá odkaz na GitHub repozitář, zjistí použitý programovací jazyk, spustí odevzdaný projekt v izolovaném kontejnerovém prostředí a uloží výsledek vyhodnocení.

Backendová aplikace byla navržena podle principů Clean Architecture a implementována na platformě .NET. Bezpečné spuštění cizího zdrojového kódu zajišťuje Podman, zatímco výsledky jsou asynchronně předávány pomocí RabbitMQ. Důležitou součástí řešení jsou samostatné runnery pro jednotlivé programovací jazyky, které umožňují systém dále rozšiřovat bez výrazných změn hlavní aplikační logiky.

Výsledkem práce je funkční prototyp, který omezuje ruční práci mentorů, sjednocuje způsob zpracování odevzdaných úloh a snižuje rizika spojená se spouštěním nedůvěryhodného kódu. Stanovený cíl práce byl naplněn, přesto existuje prostor pro zlepšení.

Conclusions

The aim of this bachelor's thesis was to design and implement the InQuest system for automated evaluation of programming assignments. The proposed solution accepts a link to a GitHub repository, detects the programming language used, runs the submitted project in an isolated container environment and saves the evaluation result.

The backend application was designed according to the principles of Clean Architecture and implemented on the .NET platform. Safe execution of untrusted source code is ensured by Podman, while the results are transmitted asynchronously using RabbitMQ. An important part of the solution is the use of separate runners for individual programming languages, which allow the system to be further expanded without significant changes to the main application logic.

The result of the thesis is a functional prototype that reduces the amount of manual work required from mentors, unifies the processing of submitted assignments and reduces the risks associated with running untrusted code. The defined objective of the thesis was achieved, though there is still room for improvement.

A TODO: Obsah elektronických dat

Na samotném konci textu práce je uveden stručný popis obsahu elektronických dat odevzdaných v systému katedry informatiky spolu s textem. Tato data jsou nedílnou součástí práce a tvoří (datovou) přílohu textu práce. Povinné položky struktury dat jsou:

text/

Adresář s textem práce ve formátu PDF, vytvořený s použitím závazného stylu KI PřF UP v Olomouci pro závěrečné práce, včetně všech (textových) příloh, a všechny soubory potřebné pro bezproblémové vytvoření PDF dokumentu textu (případně v ZIP archivu), tj. zdrojový text textu a příloh, vložené obrázky, apod.

README.*

Textový soubor (s příponou např. .txt) s informacemi o opakovatelném způsobu použití ostatních dat práce – typicky plně reprodukovatelný co nejúplnější funkční postup zprovoznění software vytvořeného v rámci práce, tzn. jeho případné instalace/nasazení a spuštění, včetně uvedení všech požadavků pro bezproblémový provoz; za zprovoznění software se nepovažuje zpřístupnění (např. po Internetu) již někde zprovozněného software.

Adresáře a soubory s veškerými ostatními autorskými daty práce (případně v ZIP archivu) – typicky spustitelné a další soubory software vytvořeného v rámci práce potřebné pro bezproblémový provoz software, případně jeho instalační program, a kompletní zdrojové texty software a další data nutná pro plně reprodukovatelné korektní vytvoření spustitelných souborů.

Dále mohou data obsahovat například:

- ukázková a testovací data použitá v práci nebo pro potřeby posouzení práce v rámci její obhajoby,
- položky bibliografie v elektronické podobě, příp. jiná relevantní literatura a dokumentace vztahující se k práci,
- cizí data (software) potřebná pro bezproblémové použití autorských dat práce (software), která nejsou standardní součástí předpokládaného (softwarového) vybavení uživatele.

U veškerých cizích obsažených materiálů jejich zahrnutí dovoluují podmínky pro jejich veřejné šíření nebo přiložený souhlas držitele práv k užití. Pro všechny použité (a citované) materiály, u kterých toto není splněno a nejsou tak obsaženy, je uveden jejich zdroj, např. webová adresa, v bibliografii nebo textu práce nebo souboru README.*.

Literatura

- [1] Edhouse s.r.o. *Edhouse: Vývoj softwaru pro globální lídry* [online]. 2026 [cit. 2026-6-15]. Dostupný z: <https://www.edhouse.eu/cz/>.
- [2] HackerRank. *HackerRank: Developer skills platform* [online]. 2026 [cit. 2026-7-29]. Dostupný z: <https://www.hackerrank.com/>.
- [3] CodeRunner. *CodeRunner: Free open-source Moodle question type* [online]. 2026 [cit. 2026-7-29]. Dostupný z: <https://coderunner.org.nz/>.
- [4] Judge0. *Judge0: Robust, scalable, and open-source code execution system* [online]. 2026 [cit. 2026-7-29]. Dostupný z: <https://judge0.com/>.
- [5] Microsoft. *.NET / Free. Cross-platform. Open source* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://dotnet.microsoft.com/en-us/>.
- [6] Getachew, Samuel. *Introduction to ASP.NET Core Web API: A Complete Guide for Developers* [online]. 2024 [cit. 2026-6-22]. Dostupný z: <https://medium.com/@solomongetachew112/introduction-to-asp-net-core-web-api-a-complete-guide-for-developers-347df16b8ab6>.
- [7] Microsoft. *Migrations Overview* [online]. 2024 [cit. 2026-6-22]. Dostupný z: <https://learn.microsoft.com/en-us/ef/core/managing-schemas/migrations/>.
- [8] Code, Bob. *Entity Framework Core (Code-First): Introduction, Best Practices, Repository Pattern & Clean Architecture* [online]. 2024 [cit. 2026-6-22]. Dostupný z: <https://medium.com/@codebob75/entity-framework-core-code-first-introduction-best-practices-repository-pattern-clean-22b6152bcb81>.
- [9] SmartBear Software. *API Documentation & Design Tools / Swagger* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://swagger.io/>.
- [10] Pranaya, Rout. *AutoMapper in C#* [online]. 2023 [cit. 2026-6-22]. Dostupný z: <https://dotnettutorials.net/lesson/autmapper-in-c-sharp/>.
- [11] Serilog Contributors. *Serilog* [online]. 2026 [cit. 2026-6-26]. Dostupný z: <https://serilog.net/>.
- [12] Red Hat, Inc. *What is a REST API?* [online]. 2020 [cit. 2026-6-22]. Dostupný z: <https://www.redhat.com/en/topics/api/what-is-a-rest-api>.
- [13] Red Hat, Inc. *What is Podman?* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://www.redhat.com/en/topics/containers/what-is-podman>.
- [14] Podman Container Tools. *Podman* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://podman.io/>.
- [15] Preston, Dan. *Keep your secrets secret with the new Podman secrets command* [online]. 2021 [cit. 2026-6-27]. Dostupný z: <https://www.redhat.com/en/blog/new-podman-secrets-command>.

- [16] Jain, Urvashi. *Podman REST API* [online]. 2022 [cit. 2026-6-27]. Dostupný z: <https://www.redhat.com/en/blog/podman-rest-api>.
- [17] Broadcom Inc. *Messaging that just works — RabbitMQ* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://www.rabbitmq.com/>.
- [18] GitLab Inc. *GitLab CI/CD documentation* [online]. 2026 [cit. 2026-6-26]. Dostupný z: <https://docs.gitlab.com/ci/>.
- [19] Sanil, Zayna. *Bash Scripting Tutorial – Linux Shell Script and Command Line for Beginners* [online]. 2023 [cit. 2026-6-26]. Dostupný z: <https://www.freecodecamp.org/news/bash-scripting-tutorial-linux-shell-script-and-command-line-for-beginners/>.
- [20] Saluja, Ashish. *Getting Started with GitHub REST API* [online]. 2026 [cit. 2026-6-22]. Dostupný z: <https://www.loginradius.com/blog/engineering/github-api>.
- [21] Martin, Robert C. *The Clean Architecture* [online]. 2012 [cit. 2026-6-22]. Dostupný z: <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>.